Hospital Twin — Simulated Hospital Environment for AI Solution Testing

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

- What this is . . . 4
- How it works . . . 5
- Target model . . . 5
- The deployment model — Assess → Design → Generate/Build → Validate → Expand . . . 5
- Benefits . . . 6
- Limitations (read before relying on results) . . . 6

Requirements 7

- 1. A defined AI-under-test integration contract . . . 7
- 2. A synthetic data source bound to the target model . . . 7
- 3. Scenario definitions for the pilot AI use cases . . . 8
- 4. A reference profile for fidelity gates . . . 8
- 5. Explicit pass/fail gates (fidelity, privacy, scenario) . . . 8
- 6. Twin scope & configuration record . . . 9

Global Requirements — Synthetic Data & AI Test-Enablement Solutions 9

- 1. A Python data/engineering environment . . . 9
- 2. A target clinical data model / standard . . . 10
- 3. An isolated non-production test environment . . . 10
- 4. Privacy, governance & compliance stakeholders and policy . . . 10
- 5. AI/QA use cases and a pilot scope . . . 10
- 6. Reference/real-data access for fidelity (read-only, minimized) . . . 11

Build Guide — Implementation 11

- Stage 1 — Assess . . . 11
- Stage 2 — Design . . . 12
- Stage 3 — Generate / Build . . . 12
- Stage 4 — Validate . . . 13
- Stage 5 — Expand . . . 13

Testing & Validation 14

- Test 1 — Twin build integrity (does it stand up?) . . . 14
- Test 2 — Scenario executes end-to-end (Gate 2) . . . 14
- Test 3 — Fidelity & coverage (fidelity gate) . . . 15
- Test 4 — Privacy validation (privacy gate) . . . 15
- Test 5 — Scenario-based AI evaluation (scenario gate) . . . 15
- Test 6 — Reproducibility . . . 15

Release signal . . . 16
Troubleshooting 16
Readiness reports a BLOCKER . . . 16
Scope config fails validation . . . 16
Simulator ABORTS on a real-PHI marker . . . 16
Simulator reports zero population or missing domains . . . 17
Scenario runner: AI plug-in never invoked / empty evaluation record (Gate 2 fail) . . . 17
Real adapter import error . . . 17
Fidelity gate NOT MET . . . 18
"Observed value not found" in the fidelity report . . . 18
Privacy check FAIL . . . 18
Non-reproducible results with the same seed . . . 18
Gate passed but production use is being assumed . . . 19
Best Practices 19
Safety & privacy (non-negotiable) . . . 19
Design . . . 19
Scenarios & evaluation . . . 20
Fidelity . . . 20
Operations & governance . . . 20
Deliverables Summary 20
Documentation . . . 20
The 5 Python tools (`scripts/`) . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary

Outcome. A **hospital twin** — a simulated hospital environment (patients, encounters, clinical workflows, and operational event dynamics) driven entirely by **synthetic clinical data** — in which clinical AI solutions are plugged in and evaluated against realistic scenarios **before** they are trusted anywhere near real patients or production systems. Teams get an evidence-backed, repeatable place to prove clinical AI safe, at a fraction of a bespoke consulting build.

The problem it solves. Testing clinical AI in production is unacceptable, yet AI, QA, and architecture teams need somewhere realistic to try it first. Without a safe sandbox, evaluation is unrepeatable and failures surface late — at the worst possible time, near real patients. The hospital twin is the digital-twin harness that closes that gap.

Approach. The solution follows a pilot-first deployment model — **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** — with explicit numeric pass/fail gates and a documented rollback. A competent data/platform engineer designs the twin's entities, workflows, and event dynamics; wires a synthetic data feed against a chosen target model (FHIR, OMOP, or the native schema); defines versioned scenarios and a thin AI plug-in contract; runs a pilot evaluation; and only then expands scope. Five Python tools do the work — readiness assessment (with a built-in scope/config helper), the twin/event simulator, the scenario runner + AI plug-in harness, a fidelity/coverage report, and a privacy/validation check.

Why it is safe. The twin runs in an **isolated, non-production** environment and is fed only by **synthetic** data — **no real PHI** ever enters the twin or its AI test runs, enforced by a built-in real-PHI leakage guard. Synthetic-data and AI frameworks are integrated at the pattern level, with vendor specifics deferred to the vendor's documentation. Every gate produces evidence, not a verdict: HIPAA/HITRUST de-identification, disclosure-risk acceptance, and IRB/privacy sign-off are **deferred to the organization's privacy, compliance, and legal functions**, and passing the twin informs — but does not replace — the human decision to authorize any production use.

Bottom line. A hospital twin where clinical AI is proven safe against realistic scenarios before it ever touches a real patient — repeatable, auditable, and iteratively refined for higher fidelity.

Solution Overview

What this is

A **hospital twin** is a simulated hospital environment — patients, encounters, clinical workflows, and operational event dynamics — driven entirely by **synthetic (artificial) clinical data**, in which AI solutions are plugged in and evaluated against defined scenarios **before** they are trusted anywhere near real patients or production systems. It is the digital-twin **test harness** that lets AI, QA, and architecture teams assess a clinical AI solution safely, repeatably, and at low cost.

This solution gives a competent data/platform engineer the exact, ordered steps to design and build such a twin, wire a synthetic data feed into it, define evaluation scenarios and AI plug-in points, run a pilot evaluation, and then expand fidelity — plus five Python tools that do the assessment, simulation, reporting, validation, and configuration work.

The twin is **not** a production system, not a data warehouse, and not a de-identification service. It is a controlled simulation used for evaluation. Real Protected Health Information (PHI) is **never** placed into the

twin or its AI test runs (see the Synthetic global requirements and the boundary note below).

How it works

The twin is composed of four layers, built in order:

1. **Entity & state model** — the simulated population and its objects: patients, encounters,

conditions, medications, labs/vitals, orders, and the operational units (beds, service lines, queues) the twin represents. Modeled against a **target clinical data model** — typically **FHIR R4** resources, **OMOP CDM** tables, or the source system's own schema — chosen with the AI/QA teams (global requirement #2).

1. **Synthetic data feed** — the source of the population and its records. Either **generated**

by a synthetic-data engine (e.g. a Synthea-style generator or a commercial platform, integrated at the pattern level) or **replayed** from a previously generated, governed synthetic dataset. The feed is what makes the twin populated and clinically plausible without real PHI.

1. **Event & workflow dynamics** — the simulation clock that advances the twin: admissions,

transfers, order placement, result availability, discharges, and the timing/queueing that make a scenario behave like a hospital rather than a static table dump.

1. **Scenario & AI plug-in harness** — named, versioned **scenarios** (a defined starting state +

an event sequence + expected/acceptable AI behavior) and the **plug-in point** where an AI solution under test is invoked (via a local adapter/callable or an isolated test endpoint) and its outputs are captured for evaluation.

An **evaluation run** feeds a scenario's events through the twin, invokes the AI at its plug-in point, records the AI's responses, and scores them against the scenario's expected behavior and against fidelity/coverage/privacy gates. The result is an evidence-backed judgment of whether the AI is ready for broader (still non-production) adoption — never an automatic production authorization, which remains a human decision.

Target model

The twin models against whatever clinical standard the AI use cases require:

• **FHIR R4** — resource-oriented (Patient, Encounter, Condition, MedicationRequest, Observation,

etc.); natural when the AI consumes FHIR.

• **OMOP CDM** — analytic, table-oriented (person, visit_occurrence, condition_occurrence,

measurement, drug_exposure); natural for population/analytic AI.

• **Native source schema** — when the AI is tied to a specific EHR/source system's shape.

The tools in this solution are **model-agnostic**: they operate on a normalized internal representation and can emit/ingest FHIR- or OMOP-shaped records. The chosen target is recorded once in the twin's scope config and reused everywhere.

The deployment model — Assess → Design → Generate/Build → Validate → Expand

Every Synthetic solution, including this one, follows the same pilot-first model with explicit pass/fail gates and a documented rollback:

• **Assess** — profile the AI use cases, the target model, the available synthetic data source, the isolated environment, and governance. Produce a readiness assessment (`Twin Readiness – Pilot – v1`). Gate: readiness accepted.

• **Design** — design the twin's entities, workflows, event dynamics, and the scenario/AI plug-in contract. Produce the twin design (`Hospital Twin Design – Pilot – v1`).

• **Generate/Build** — stand up the twin in the isolated environment, wire the synthetic feed, and implement the scenario suite (`Twin Scenario Suite – Pilot – v1`). Gate: twin runs and a scenario executes end-to-end.

• **Validate** — run the pilot evaluation: fidelity/coverage of the twin, privacy validation of the synthetic data, and the scenario-based AI evaluation. Produce the fidelity report (`Twin Fidelity Report – Pilot – v1`) and the evaluation record. Gate: all pass/fail gates met.

• **Expand** — only after the pilot passes, add service lines, scenarios, and fidelity depth (`Hospital Twin Design – Cardiology – v1`, etc.), re-validating at each step.

Rollback: the twin lives only in the isolated non-production environment; if a gate fails, stop, retain the evidence, and either refine the twin/scenario or defer the AI — nothing has touched production or real PHI, so rollback is discarding the run and its synthetic artifacts.

Benefits

• **Safe** — clinical AI is exercised against realistic dynamics with **zero** real-patient risk; no real PHI enters the twin.

• **Repeatable** — scenarios are versioned and deterministic (seeded), so an AI can be re-tested identically across versions and regressions caught.

• **Evidence-based** — every evaluation yields a scored record and a fidelity/coverage report you can show to architecture, QA, and governance.

• **Low cost & self-guided** — a fraction of a bespoke consulting build; a competent engineer can run the pilot from these steps and tools.

Limitations (read before relying on results)

• A twin is a **model of** a hospital, not the hospital. Passing in the twin is **necessary but not sufficient** for production trust; it does not by itself authorize production use.

• **Fidelity is bounded** by the synthetic data source and the modeled dynamics. Claims of realism are only as strong as the reference distributions used to validate them (global requirement #6).

• The twin does **not** make regulatory or de-identification determinations. HIPAA/HITRUST

de-identification, IRB/privacy sign-off, and disclosure-risk acceptance are **deferred** to the organization's privacy, compliance, and legal functions — this solution supplies technical method and evidence only.

• Vendor synthetic-data/AI frameworks are integrated at the **pattern** level; version-specific

console internals must be verified against the vendor's own documentation and your version.

Requirements

Before building the hospital twin, satisfy the **six category-wide requirements** in `global-requirements-synthetic.md` (a Python data/engineering environment; a target clinical data model/standard; an isolated non-production test environment; privacy/governance/compliance stakeholders and policy; AI/QA use cases and a pilot scope; and read-only, minimized reference/real-data access for fidelity). Those are assumed here and are **not** repeated.

The following prerequisites are **specific to the hospital twin**. Each names *what* it is, *why* it is required, *how to verify* it, and *how to implement* it if missing. Confirm all of them before starting the Build Guide (`03-implementation.md`).

1. A defined AI-under-test integration contract

• **Description:** A documented contract for how the AI solution under test is invoked from the

twin — the **plug-in point**: input shape (e.g. a FHIR Bundle, an OMOP record set, or a feature vector), output shape (e.g. a risk score, an alert, a recommendation), and the invocation mechanism (a local Python callable/adapter, or an **isolated** test endpoint). Recorded as the `Twin AI Plug-In Contract – Pilot – v1`.

• **Why required:** Scenarios and the plug-in harness are built against this contract; without it

the twin cannot call the AI or capture its outputs for scoring.

• **How to verify:** Confirm the AI team can supply (a) an input schema, (b) an output schema, and

(c) a callable/endpoint that runs **inside the isolated environment** with no production or real-PHI dependency.

• **How to implement if missing:** Work with the AI team to wrap the model behind a thin adapter

that accepts the twin's normalized input and returns a structured output; document the two schemas before building scenarios.

2. A synthetic data source bound to the target model

• **Description:** A working **synthetic data source** — a generator (Synthea-style or a commercial

platform, at the pattern level) or a previously generated governed synthetic dataset — that produces records in, or mappable to, the chosen target model (FHIR/OMOP/native). Recorded as the `Synthetic Feed Binding – Pilot – v1`.

• **Why required:** The twin's population and records come from this feed; it is what lets the twin

be populated and clinically plausible with **no real PHI**.

• **How to verify:** Generate (or load) a small sample and confirm it parses into the twin's target

model and that the in-scope domains (patients, encounters, conditions, meds, labs/vitals) are present.

• **How to implement if missing:** Stand up the generator or obtain the governed synthetic dataset; write/confirm the mapping from its output to the target model.

3. Scenario definitions for the pilot AI use cases

• **Description:** For each pilot AI use case, at least one **scenario**: a named starting state, an ordered event sequence, and the **expected/acceptable AI behavior** (including the failure behavior that must be caught). Recorded collectively as the `Twin Scenario Suite – Pilot – v1`.

• **Why required:** A scenario is the unit of evaluation; without expected behavior there is no pass/fail signal for the AI.

• **How to verify:** Confirm each pilot use case has ≥1 scenario with an explicit expected outcome and an explicit unacceptable outcome.

• **How to implement if missing:** Run a scenario-design working session with the AI/QA and clinical SME stakeholders; capture each scenario in the twin's scenario format (see `03-implementation.md`).

4. A reference profile for fidelity gates

• **Description:** The agreed **reference distributions/statistics** the twin's synthetic output is measured against (from the read-only, minimized reference of global requirement #6, or agreed published reference distributions). Recorded as `Twin Reference Profile – Pilot – v1`.

• **Why required:** Fidelity/coverage gates compare the twin's population and events to reality; without a reference, "realistic" is unfalsifiable.

• **How to verify:** Confirm the reference source, the specific metrics in scope (e.g. age/sex distribution, condition prevalence, encounter length-of-stay, event timing), and the acceptance thresholds.

• **How to implement if missing:** Agree the reference approach (governed real stats vs. published reference distributions) and record the metrics + thresholds before validating.

5. Explicit pass/fail gates (fidelity, privacy, scenario)

• **Description:** Written, numeric **acceptance gates** for the pilot: a **fidelity** gate (how close to the reference profile is acceptable), a **privacy** gate (the disclosure-risk checks the synthetic data must pass), and a **scenario** gate (what fraction of scenarios the AI must handle acceptably). Recorded as `Twin Acceptance Gates – Pilot – v1`.

• **Why required:** The deployment model requires explicit pass/fail gates and a documented rollback; the tools emit signals that only mean something against agreed thresholds.

• **How to verify:** Confirm each gate has a metric, a threshold, and a named owner who accepts it.

• **How to implement if missing:** Define the three gates with the AI/QA and privacy stakeholders

before the pilot evaluation; the privacy gate's *acceptance* is deferred to the privacy office.

6. Twin scope & configuration record

• **Description:** A single **scope/config record** for the pilot twin — target model, in-scope domains, service line(s), population size, simulation clock settings, the AI plug-in contract reference, and the seed for reproducibility. Recorded as `Twin Scope Config – Pilot – v1` (produced/validated by the config helper tool).

• **Why required:** Every tool and step reads this one record; it is the reproducibility and traceability anchor for the whole twin.

• **How to verify:** Confirm the config validates against the config helper (`twin_readiness_assess.py --validate-config`) with no errors.

• **How to implement if missing:** Generate a starter config with the config helper (`twin_readiness_assess.py --init-config`), fill it in, and validate.

Global Requirements — Synthetic Data & AI Test-Enablement Solutions

These requirements apply to **every** solution in the **Synthetic** category (synthetic clinical data, hospital-twin simulation, and safe AI test enablement). Verify each before starting a solution's steps.

> ***Safety, vendor & regulatory boundary (applies to every Synthetic solution).*** *These solutions generate and validate **synthetic** (artificial) data and run in **isolated non-production** test environments. **Never place real Protected Health Information (PHI) into a shared synthetic harness, twin, or AI test environment** — where real data is touched at all it is read-only, minimized, and handled under the governing privacy controls. **Synthetic-data and digital-twin tools** (e.g. Synthea, and commercial platforms) and **AI/model frameworks** are integrated at the pattern/architectural level; specific vendor console internals are **cited or deferred** to that vendor's documentation, never invented. **Regulatory and legal interpretation** (HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, disclosure risk acceptance, and any ITAR/NERC/NRC or contract constraints) is **deferred to the organization's privacy, compliance, and legal functions** — the solutions provide the technical method and evidence, not legal determinations.*

1. A Python data/engineering environment

• **Description:** A supported **Python** environment (e.g. 3.10+) with the standard data toolchain (data frames, statistics, and the synthetic-data / validation libraries the solution uses).

• **Why required:** The automation and reusable assets are authored in Python (generation, de-identification, fidelity/utility metrics, scenario runners).

• **How to verify:** Confirm Python + the required packages install and run in the target environment.

• **How to implement if missing:** Provision the environment and install dependencies before starting.

2. A target clinical data model / standard

• **Description:** A defined **clinical data model or interoperability standard** to generate against — e.g. **FHIR**, **OMOP CDM**, or the source system's own schema — plus the fields/domains in scope (patients, encounters, conditions, medications, labs, vitals, notes, etc.).

• **Why required:** Realistic synthetic data and a credible hospital twin require a target schema and clinically plausible relationships; validation is measured against it.

• **How to verify:** Confirm the target model/standard and the in-scope domains with the AI/QA teams.

• **How to implement if missing:** Select and document the target model before generating.

3. An isolated non-production test environment

• **Description:** A **segregated, non-production** environment (compute + storage) for the synthetic harness, hospital twin, and AI test runs — separate from production and from real-PHI systems.

• **Why required:** Safe evaluation of AI solutions before broader adoption depends on isolation; it also bounds any residual disclosure risk.

• **How to verify:** Confirm the environment is isolated, access-controlled, and not connected to production PHI stores.

• **How to implement if missing:** Stand up / obtain the isolated environment before running.

4. Privacy, governance & compliance stakeholders and policy

• **Description:** The governing **privacy, data-governance, and compliance** policies and the named stakeholders (privacy officer, security, and — where real source data or de-identification claims are involved — IRB/legal) whose requirements the work must satisfy.

• **Why required:** Regulated healthcare data constraints, de-identification acceptance, and enterprise data governance are policy decisions the technical method must align to, not defaults.

• **How to verify:** Confirm the applicable policies, disclosure-risk threshold, and approvers.

• **How to implement if missing:** Identify the policies and stakeholders before generating or releasing test data.

5. AI/QA use cases and a pilot scope

• **Description:** The **AI use cases and QA scenarios** the synthetic data / twin must support, and a **pilot scope** to validate on before broader rollout.

• **Why required:** Fidelity, coverage, and scenario design are driven by the AI use cases; every solution follows an **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model proven on a pilot.

• **How to verify:** Confirm the target AI use cases, evaluation goals, and a named pilot.

• **How to implement if missing:** Elicit the use cases and define the pilot before building the harness.

6. Reference/real-data access for fidelity (read-only, minimized)

• **Description:** Where synthetic **fidelity/utility** must be measured against reality, **read-only, minimized** access to reference statistics or a governed real dataset (under the privacy controls in §4) — or agreed reference distributions where real data cannot be used.

• **Why required:** Fidelity and validation assets compare synthetic output to real-world distributions and relationships; without a reference the claims are unfounded.

• **How to verify:** Confirm the reference source and its access/governance terms.

• **How to implement if missing:** Agree the reference approach (governed real stats vs. published reference distributions) before validating.

*The deployment model across every Synthetic solution is the same: **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, with explicit pass/fail (fidelity/privacy) gates and a documented rollback. Automation and reusable assets are delivered as **Python** — never real PHI is written to a shared harness, twin, or AI test environment.*

Build Guide — Implementation

Build the hospital twin by following the house deployment model **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates. Every artifact is **named** with the `<Purpose> – <Scope> – <Stage>` convention and that exact name is reused in testing, troubleshooting, and the tools. Do all of this in the **isolated non-production environment** (global requirement #3). **Never** load real PHI into the twin.

The five Python tools referenced below live in `scripts/`; run any with `--help`. All are safe-by-default (dry-run where they write).

Stage 1 — Assess

Goal: confirm readiness and produce the `Twin Readiness – Pilot – v1` assessment.

1. **Confirm the six global requirements** and the six solution-specific requirements (`02-requirements.md`) are met. Record the target model (FHIR/OMOP/native), the pilot AI use cases, and the named pilot service line.

1. **Run the readiness assessment tool** against your environment and inputs:

```
python scripts/twin_readiness_assess.py --model fhir --use-cases sepsis-alert \
    --output "Twin Readiness - Pilot - v1.json"
```

It profiles the declared inputs (Python env, target model, synthetic source reachability, AI plug-in contract presence, scenario count, reference profile presence, gates presence) and writes a readiness score with any blockers. It is **read-only** and writes only the report.

1. **Resolve blockers.** Any `BLOCKER` in the readiness report must be cleared before Stage 2.

GATE 1 (readiness): `Twin Readiness – Pilot – v1` shows no blockers and readiness ≥ the agreed threshold. If not, stop and remediate.

Stage 2 — Design

Goal: produce `Hospital Twin Design – Pilot – v1` and the AI plug-in contract.

1. **Define the entity & state model.** List the twin's entities against the target model

(Patient/person, Encounter/visit, Condition, Medication, Observation/labs+vitals, Order), and the operational units for the pilot service line (beds, queues, service line). Keep the pilot **narrow** — one service line, one or two AI use cases.

1. **Design the event & workflow dynamics.** Define the events the simulation clock emits

(admit → order → result → medicate → transfer → discharge) and their timing/queueing. Decide the clock granularity (e.g. 1 simulated hour per tick) and the pilot horizon (e.g. 7 simulated days).

1. **Author the AI plug-in contract** (`Twin AI Plug-In Contract – Pilot – v1`): input shape,

output shape, and the isolated invocation mechanism (local callable/adapter or isolated endpoint). The twin's harness calls the AI through this contract only.

1. **Initialize and validate the scope config** with the config helper built into the readiness

tool:

```
python scripts/twin_readiness_assess.py --init-config --model fhir \
    --service-line "pilot-medsurg" --population 500 --seed 42 \
    --out "Twin Scope Config - Pilot - v1.json"
python scripts/twin_readiness_assess.py --validate-config "Twin Scope Config - Pilot - v1.json"
```

Fill in the AI plug-in contract reference and the in-scope domains, then re-validate until it passes. This `Twin Scope Config – Pilot – v1` is read by every later tool.

Stage 3 — Generate / Build

Goal: stand up the twin, wire the synthetic feed, implement the scenario suite; prove one scenario runs end-to-end.

1. **Wire the synthetic data feed** (`Synthetic Feed Binding – Pilot – v1`). Generate or load the

pilot population from your synthetic source into the target model. Confirm **no real PHI** is present — the source is a synthetic generator or a governed synthetic dataset only.

1. **Load the population into the twin.** Point the simulator at the scope config and the feed:

```
python scripts/twin_simulator.py --config "Twin Scope Config - Pilot - v1.json" \
    --feed "Synthetic Feed Binding - Pilot - v1" --dry-run
```

`--dry-run` (default) plans the run and reports population/event counts **without** writing the run log. Review the plan.

1. **Author the scenario suite** (`Twin Scenario Suite – Pilot – v1`). For each pilot use case,

write a scenario file: a starting state selector, an ordered event sequence, and the expected/unacceptable AI behavior. Keep them versioned alongside the scope config.

1. **Execute one scenario end-to-end** with the scenario runner + AI plug-in harness, using a

built-in mock AI adapter first to prove the wiring, then the real AI adapter:

```
python scripts/twin_scenario_runner.py --config "Twin Scope Config - Pilot - v1.json" \
    --scenario "Twin Scenario Suite - Pilot - v1/sepsis-alert.json" \
    --ai-adapter mock --commit
```

`--commit` writes the evaluation record (default is dry-run). The harness advances the twin through the scenario's events, invokes the AI at each plug-in point, and captures its outputs.

GATE 2 (end-to-end): the twin runs, the scenario executes, and the AI plug-in is invoked and captured — proven by a non-empty `Twin Evaluation Record – Pilot – v1`. If not, stop (see `06-troubleshooting.md`).

Stage 4 — Validate

Goal: run the pilot evaluation and clear the fidelity, privacy, and scenario gates.

1. **Fidelity & coverage.** Compare the twin's synthetic population and events to the reference

profile:

```
python scripts/twin_fidelity_report.py --config "Twin Scope Config - Pilot - v1.json" \
    --reference "Twin Reference Profile - Pilot - v1.json" \
    --out "Twin Fidelity Report - Pilot - v1.html"
```

Produces the `Twin Fidelity Report – Pilot – v1` with per-metric distances vs. thresholds and a scenario-coverage summary.

1. **Privacy validation.** Run the disclosure-risk checks on the synthetic data feeding the twin:

```
python scripts/twin_privacy_validate.py --config "Twin Scope Config - Pilot - v1.json" \
    --dataset "Synthetic Feed Binding - Pilot - v1" \
    --out "Twin Privacy Validation - Pilot - v1.json"
```

It runs technical checks (exact-match/uniqueness against any reference, quasi-identifier k-anonymity, and a real-PHI leakage guard) and reports PASS/FAIL **per check**. The **acceptance** of residual disclosure risk is **deferred to the privacy office** — the tool supplies evidence, not the determination.

1. **Scenario-based AI evaluation.** Run the full scenario suite against the **real** AI adapter

and score it:

```
python scripts/twin_scenario_runner.py --config "Twin Scope Config - Pilot - v1.json" \
    --scenario-suite "Twin Scenario Suite - Pilot - v1" --ai-adapter real --commit \
    --out "Twin Evaluation Record - Pilot - v1.json"
```

GATE 3 (acceptance): all three gates in `Twin Acceptance Gates – Pilot – v1` are met — fidelity within threshold, every privacy check PASS (with privacy-office acceptance of residual risk), and the scenario pass-rate ≥ threshold. **Rollback if any gate fails:** stop, retain the evidence, refine the twin/scenario or defer the AI; nothing has touched production or real PHI, so rollback is discarding the run's synthetic artifacts.

Stage 5 — Expand

Goal: grow the twin only after the pilot passes.

1. **Add scope incrementally.** Clone the pilot design to a new service line — e.g.

`Hospital Twin Design – Cardiology – v1` with `Twin Scope Config – Cardiology – v1` — adding entities, scenarios, and fidelity depth.

1. **Re-validate at each step.** Re-run fidelity, privacy, and scenario gates for the new scope

(`Twin Fidelity Report – Cardiology – v1`, etc.). Never expand past a failing gate.

1. **Record the twin's provenance.** Keep every scope config, scenario suite, evaluation record,

and report versioned so any AI evaluation is reproducible and auditable.

Production note: passing the twin authorizes **further evaluation**, not production use. Production authorization for clinical AI is a human decision by the accountable owners, informed by — not replaced by — the twin's evidence.

Testing & Validation

These procedures validate the hospital twin and the AI evaluation it produces. Run them against the named artifacts from `03-implementation.md` (the `– Pilot – v1` set). Each has an explicit **pass/fail**. The three acceptance gates in `Twin Acceptance Gates – Pilot – v1` are the release signal for the pilot.

> *All validation runs in the **isolated non-production environment**. Privacy-check **acceptance** is a privacy-office determination; the tools provide evidence only.*

Test 1 — Twin build integrity (does it stand up?)

• **Procedure:** Run the simulator in dry-run against the config and feed:

```
python scripts/twin_simulator.py --config "Twin Scope Config - Pilot - v1.json" --feed "Synthetic Feed Binding - Pilot - v1" --dry-run
```

.

• **Expected PASS:** The plan reports the configured population size (±0), all in-scope domains

present, a valid event plan, and **no real-PHI markers** detected in the feed.

• **FAIL:** Missing domains, zero population, an unparseable feed, or any real-PHI marker. Do not

proceed — see `06-troubleshooting.md`.

Test 2 — Scenario executes end-to-end (Gate 2)

• **Procedure:** Run one scenario with the **mock** AI adapter and `--commit`:

```
python scripts/twin_scenario_runner.py --config "Twin Scope Config - Pilot - v1.json" --scenario "Twin Scenario Suite - Pilot - v1/sepsis-alert.json" --ai-adapter mock --commit
```

.

• **Expected PASS:** The twin advances through every scenario event, the AI plug-in is invoked at

each plug-in point, outputs are captured, and a non-empty `Twin Evaluation Record – Pilot – v1` is written.

• **FAIL:** The run aborts, the plug-in is never invoked, or the record is empty. This is **Gate 2**

— fix before validation.

Test 3 — Fidelity & coverage (fidelity gate)

• **Procedure:** `python scripts/twin_fidelity_report.py --config "Twin Scope Config - Pilot - v1.json" --reference "Twin Reference Profile - Pilot - v1.json" --out "Twin Fidelity Report - Pilot - v1.html"`.

• **Expected PASS:** Every in-scope metric's distance to the reference profile is **within** its

threshold in `Twin Acceptance Gates – Pilot – v1`, and scenario coverage meets the agreed minimum (every pilot use case exercised by ≥1 scenario).

• **FAIL:** Any metric exceeds its threshold, or a use case is uncovered. Refine the synthetic feed

or dynamics and re-run; do not accept an out-of-threshold twin.

Test 4 — Privacy validation (privacy gate)

• **Procedure:** `python scripts/twin_privacy_validate.py --config "Twin Scope Config - Pilot - v1.json" --dataset "Synthetic Feed Binding - Pilot - v1" --out "Twin Privacy Validation - Pilot - v1.json"`.

• **Expected PASS (technical):** Every check reports PASS — no exact-match records against any

reference, quasi-identifier k-anonymity ≥ the configured *k*, and the **real-PHI leakage guard** finds no real-PHI markers. The report is then handed to the privacy office.

• **FAIL:** Any check FAIL — investigate and remediate the synthetic source before use. **A

technical PASS is not an acceptance:** residual-disclosure-risk acceptance is the privacy office's determination.

Test 5 — Scenario-based AI evaluation (scenario gate)

• **Procedure:** Run the full suite against the **real** AI adapter:

```
python scripts/twin_scenario_runner.py --config "Twin Scope Config - Pilot - v1.json" --scenario-suite "Twin Scenario Suite - Pilot - v1" --ai-adapter real --commit --out "Twin Evaluation Record - Pilot - v1.json".
```

• **Expected PASS:** The AI produces the expected/acceptable behavior for at least the threshold

fraction of scenarios in `Twin Acceptance Gates – Pilot – v1`, and produces **no** unacceptable behavior on any scenario flagged critical.

• **FAIL:** Pass-rate below threshold, or any critical scenario yields the unacceptable behavior.

The AI is **not** ready for broader adoption; return findings to the AI team.

Test 6 — Reproducibility

• **Procedure:** Re-run Test 2 with the **same seed** from `Twin Scope Config – Pilot – v1`.

• **Expected PASS:** Identical population and event sequence, and an identical evaluation record for a deterministic AI adapter (mock) — proving the twin is reproducible for regression testing.

• **FAIL:** Divergent results with the same seed — a non-determinism bug; fix before relying on regression comparisons.

Release signal

The pilot is **validated** only when Tests 1–6 PASS **and** all three gates in `Twin Acceptance Gates – Pilot – v1` are met (fidelity within threshold, every privacy check PASS with privacy-office acceptance, scenario pass-rate ≥ threshold). Only then proceed to **Expand**.

Troubleshooting

Common failure modes when building and running the hospital twin, with resolutions. Artifacts are referenced by their `<Purpose> – <Scope> – <Stage>` names from `03-implementation.md`.

Readiness reports a BLOCKER

• **Symptom:** `twin_readiness_assess.py` prints `NOT READY` with one or more `BLOCKER` findings and exits non-zero.

• **Cause:** A required input is missing — no synthetic source, no AI plug-in contract, no AI use cases, or Python < 3.10.

• **Resolution:** Clear each blocker named in `Twin Readiness – Pilot – v1` (bind the `Synthetic Feed Binding – Pilot – v1`, author the `Twin AI Plug-In Contract – Pilot – v1`, declare use cases, or upgrade Python). Re-run until `READY`. Do not proceed to Design with blockers.

Scope config fails validation

• **Symptom:** `twin_readiness_assess.py --validate-config` prints `FAIL` with a list of errors.

• **Cause:** A missing key, a non-positive `population`, a non-integer `seed`, an unsupported `target_model`, or empty `in_scope_domains`.

• **Resolution:** Fix the listed fields in `Twin Scope Config – Pilot – v1.json`. Regenerate a clean starter with `--init-config` if the file drifted, then re-validate. Every later tool reads this file, so it must pass first.

Simulator ABORTS on a real-PHI marker

• **Symptom:** `twin_simulator.py` prints `ABORT: real-PHI markers detected in the feed` and exits

code 3.

• **Cause:** The `Synthetic Feed Binding – Pilot – v1` contains real-PHI markers (e.g. `ssn`, `mrn_real`) — real data leaked into a feed that must be synthetic-only.

• **Resolution: Stop.** Do not force the run. Remove the real data from the source; regenerate the feed from the synthetic generator or a governed synthetic dataset only. Never load real PHI into the twin. Re-run the simulator and confirm `real-PHI guard: PASS`.

Simulator reports zero population or missing domains

• **Symptom:** Dry-run plan shows population 0 or absent in-scope domains.

• **Cause:** The feed did not load, or the config's `population`/`in_scope_domains` don't match the feed.

• **Resolution:** Confirm `Synthetic Feed Binding – Pilot – v1` path is correct and parses into the target model; reconcile `population` and `in_scope_domains` in `Twin Scope Config – Pilot – v1`. Re-run the dry-run (Test 1) before committing.

Scenario runner: AI plug-in never invoked / empty evaluation record (Gate 2 fail)

• **Symptom:** `twin_scenario_runner.py` runs but `points_matched` is 0 for all scenarios, or the record is empty.

• **Cause:** The scenario has no `plugin_points`, or (real adapter) `--adapter-spec` is wrong so the AI is never actually called.

• **Resolution:** First prove wiring with `--ai-adapter mock` (it echoes the scenario's `expected` label, so a correctly authored scenario passes). If mock passes but real fails, fix the `module.path:function` in `--adapter-spec` and confirm the adapter runs **inside the isolated environment** against synthetic data only. A non-empty `Twin Evaluation Record – Pilot – v1` is Gate 2.

Real adapter import error

• **Symptom:** `ERROR loading real adapter: ...` (bad module/function, or the callable raises).

• **Cause:** `--adapter-spec` not in `module.path:function` form, module not on `PYTHONPATH`, or the AI callable errors on the twin's input shape.

• **Resolution:** Verify the adapter contract matches `Twin AI Plug-In Contract – Pilot – v1` (input and output shapes). Ensure the module is importable from the run directory. Keep the AI behind a thin adapter so twin↔AI shape mismatches are fixed in one place.

Fidelity gate NOT MET

• **Symptom:** `twin_fidelity_report.py` marks one or more metrics `FAIL` (distance over threshold),

or reports uncovered use cases.

• **Cause:** The synthetic population/events diverge from the `Twin Reference Profile – Pilot – v1`

beyond the agreed threshold, or a pilot use case has no scenario.

• **Resolution:** Tune the synthetic feed or event dynamics toward the reference and re-run; or, if

the threshold is genuinely too tight, re-agree it with the metric owner in `Twin Acceptance Gates – Pilot – v1`. Author a scenario for any uncovered use case. Never accept an out-of-threshold twin silently.

"Observed value not found" in the fidelity report

• **Symptom:** A metric row shows `observed: None` / `observed value not found`.

• **Cause:** The reference profile's `observed_key` path doesn't exist in the run log, or no

`--run-log` was supplied.

• **Resolution:** Pass the committed `Twin Simulation Run – Pilot – v1.json` via `--run-log`, and

align each metric's `observed_key` to a real path in that run log (e.g. `event_counts.admit`).

Privacy check FAIL

• **Symptom:** `twin_privacy_validate.py` reports a `FAIL` on the leakage guard, exact-match, or

k-anonymity check.

• **Cause:** Real-PHI markers present; a synthetic record exactly copies a reference record; or a

quasi-identifier combination occurs fewer than *k* times.

• **Resolution:** Remediate the synthetic source (remove markers; increase generation diversity;

generalize/suppress rare quasi-identifiers or raise the population). Re-run until every check PASSes, then hand `Twin Privacy Validation – Pilot – v1` to the privacy office — **the technical PASS is evidence, not acceptance**; residual-risk acceptance is their determination.

Non-reproducible results with the same seed

• **Symptom:** Re-running with the same `seed` yields a different population or event stream (Test 6

fail).

• **Cause:** A non-seeded source of randomness, or the config's `seed` changed between runs.

• **Resolution:** Confirm `seed` in `Twin Scope Config – Pilot – v1` is fixed and that any custom

adapter is deterministic for the mock path. Reproducibility is what makes the twin a regression harness — fix it before comparing AI versions.

Gate passed but production use is being assumed

- **Symptom:** Stakeholders read a passing `Twin Evaluation Record – Pilot – v1` as authorization to

deploy the AI to production.

- **Cause:** Misreading twin evidence as a production sign-off.
- **Resolution:** Restate the boundary: passing the twin is **necessary but not sufficient** and

informs — does not replace — the human decision to authorize production. Production trust also requires the accountable owners' sign-off and post-rollout monitoring.

Best Practices

Durable practices for building, running, and expanding the hospital twin.

Safety & privacy (non-negotiable)

- **Never load real PHI into the twin.** The twin and its AI test runs are fed only by a synthetic

generator or a governed synthetic dataset. Where real data is touched at all (e.g. building a reference profile), it stays **read-only, minimized, and under the org's privacy controls** — never copied into the twin. Keep the `twin_privacy_validate.py` real-PHI leakage guard in every pipeline.

- **Keep the environment isolated.** Run everything in the segregated non-production environment

with no path to production PHI stores. Access-control the twin's artifacts like the sensitive test material they are.

- **Defer the determinations.** The tools give technical method and evidence. De-identification

claims, disclosure-risk acceptance, and IRB/privacy sign-off belong to privacy/compliance/legal — cite that boundary in every report you hand over.

Design

- **Start narrow, pilot-first.** One service line, one or two AI use cases, a small population.

Prove the pilot end-to-end before adding scope; breadth without a passing pilot is wasted work.

- **Model against a real standard.** Pin the target model (FHIR/OMOP/native) once in the scope

config and generate/validate against it — don't invent an ad-hoc schema the AI won't see in reality.

- **Separate the AI behind a thin adapter.** The twin calls the AI only through the plug-in

contract. This keeps the AI swappable, keeps the twin model-agnostic, and prevents the twin from depending on any one model's internals.

Scenarios & evaluation

• **Every scenario has an unacceptable outcome, not just an expected one.** The point of the twin is to *catch* bad AI behavior; a scenario with no failure condition tests nothing.

• **Version scenarios with the config.** Keep the scenario suite, scope config, reference profile, and gates versioned together so any evaluation is reproducible and diffable across AI versions.

• **Seed everything.** Use the config's seed so populations and event sequences are deterministic; reproducibility is what makes the twin a regression harness, not a one-off demo.

• **Score against agreed gates, not vibes.** Fidelity, privacy, and scenario gates are numeric and owner-accepted before the run — decide thresholds up front so results are not negotiated after the fact.

Fidelity

• **Validate realism against a reference, or state the limit.** Fidelity claims are only as strong as the reference profile. Where no reference is available, say so and treat the twin as a *functional* harness, not a *statistically representative* one.

• **Refine iteratively.** Treat fidelity as a dial you turn up over successive versions (`– v1`, `– v2`), re-validating each time, rather than chasing perfection before the first evaluation.

Operations & governance

• **Keep provenance.** Retain every scope config, scenario suite, evaluation record, and report, versioned, so an AI's twin evidence is auditable.

• **Passing the twin ≠ production authorization.** State this explicitly in every evaluation record. Production trust is a human decision informed by the twin, plus real-world monitoring after any controlled rollout.

• **Pin your tool/vendor versions.** Synthetic-data and AI frameworks change; record the versions used and re-verify vendor-specific behavior against your version's documentation before relying on it.

Deliverables Summary

What the buyer gets — one page. Everything is self-guided and safe by default: no real PHI, isolated non-production use, Python automation. Regulatory/de-identification determinations are deferred to your privacy/compliance/legal functions.

Documentation

• **Solution Overview** (`01-solution-overview.md`) — the hospital-twin concept: what it simulates (patients, encounters, workflows, events), how synthetic data drives it, how AI plugs in for evaluation, the target model, benefits/limitations, and the Assess→Design→Generate/Build→ Validate→Expand model.

• **Requirements Guide** (`02-requirements.md`) — the six solution-specific prerequisites (AI

plug-in contract, synthetic data source, scenario definitions, reference profile, pass/fail gates, scope config), each with verify/implement steps, on top of the six category-wide requirements.

• **Build Guide** (`03-implementation.md`) — exact ordered steps to design, build, wire, run a pilot evaluation, and expand the twin, with every artifact named and the three acceptance gates.

• **Testing & Validation** (`05-testing.md`) — six tests (build integrity, end-to-end scenario, fidelity/coverage, privacy, AI evaluation, reproducibility), each with pass/fail.

• **Troubleshooting** (`06-troubleshooting.md`) — common failure modes and resolutions.

• **Best Practices** (`07-best-practices.md`) — durable practices for safety, design, scenarios, fidelity, and governance.

• **Executive Summary** (`executive-summary.md`) — the outcome, approach, and why it is safe.

The 5 Python tools (`scripts/`)

All are Python 3.10+, standard-library-first, `--help`-documented, and **safe-by-default** (dry-run where they write; no real PHI to shared locations; no invented network endpoints).

1. `twin_readiness_assess.py` — *Assessment/profiling.* Profiles your inputs (Python env, target model, synthetic-source reachability, AI plug-in contract, scenarios, reference profile, gates) and writes a readiness score with blockers → `Twin Readiness – Pilot – v1`.

1. `twin_simulator.py` — *Twin/event simulator engine.* Loads the synthetic population against the scope config and advances the simulation clock (admit → order → result → medicate → transfer → discharge), emitting the event stream the twin runs on. Dry-run by default.

1. `twin_scenario_runner.py` — *Scenario runner + AI plug-in harness.* Runs a scenario (or the whole suite) through the twin, invokes the AI at each plug-in point (mock or real adapter), captures outputs, and scores them → `Twin Evaluation Record – Pilot – v1`. Dry-run by default.

1. `twin_fidelity_report.py` — *Fidelity/behavior report.* Compares the twin's synthetic population and events to the reference profile, per-metric vs. threshold, plus scenario coverage → `Twin Fidelity Report – Pilot – v1` (HTML).

1. `twin_privacy_validate.py` — *Privacy/validation check.* Runs disclosure-risk checks on the synthetic feed (exact-match/uniqueness, quasi-identifier k-anonymity, real-PHI leakage guard), PASS/FAIL per check → `Twin Privacy Validation – Pilot – v1`. Acceptance deferred to the privacy office.

The **scope/config helper** is built into the toolset: `twin_readiness_assess.py --init-config` generates and `--validate-config` checks the `Twin Scope Config – Pilot – v1` record that every tool reads, so no separate script is needed.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

twin_fidelity_report.py

twin_privacy_validate.py

twin_readiness_assess.py

twin_scenario_runner.py

twin_simulator.py